

N.O.R.A.

Neural Observer & Reasoning Architecture

Análisis Doctoral del Sistema de Observación Cognitiva

SOFIAA OS · Sistema Cognitivo Autónomo

Sprints: I-0 · F-3 · H-2 · H-3 · M-3 · D-D · G-1/G-2/G-3 · T2-6
2026 — Clasificación: Arquitectura Interna Confidencial

1. Resumen Ejecutivo

N.O.R.A. es la capa de observación, razonamiento y autoconciencia del ecosistema SOFIAA OS. Implementada a lo largo de múltiples sprints, N.O.R.A. no es un módulo único sino un sistema distribuido de sensores cognitivos que registran, interpretan y adaptan el comportamiento de SOFIAA en tiempo real.

El sistema opera en dos dimensiones paralelas:

- ▶ Dimensión de Telemetría (Sprint I-0, F-3, H-3): observa cada interacción de pipeline — latencia, tipo de tarea, capa de caché, proveedor LLM — y las persiste en Firestore para análisis longitudinal.
- ▶ Dimensión Cognitiva (Sprint M-3, D-D): construye un modelo mental del usuario que evoluciona sesión a sesión: preferencia de profundidad, formalidad, afinidades temáticas.

Adicionalmente, N.O.R.A. extiende su alcance al módulo TEC Bii como motor de reflexión institucional (Sprint T2-6), generando narrativas de estado operacional con patrones detectados y recomendaciones estratégicas.

Premisa doctoral: N.O.R.A. materializa el concepto de metacognición aplicada a sistemas de IA: un AI que se observa a sí mismo y a su usuario para mejorar iterativamente su comportamiento sin intervención humana.

2. Visión del Sistema: Las Dos Encarnaciones de N.O.R.A.

A lo largo del desarrollo de SOFIAA OS, N.O.R.A. evolucionó y se especializó en dos encarnaciones distintas pero complementarias:

2.1 Sprint I-0 — Núcleo de Observación y Análisis

La primera encarnación de N.O.R.A. es el panel de telemetría administrativa. Implementado como NoraPanel.tsx, este componente React permite al administrador del sistema observar en tiempo real el comportamiento del pipeline de SOFIAA para todos los usuarios registrados.

Archivo	src/components/admin/NoraPanel.tsx
Sprint	I-0 — N.O.R.A. Observer Panel
Tipo	React Component (Admin Panel)
Activación	Frase secreta 'Fiat lux' en page.tsx

2.2 Sprint T2-6 — Neural Observer & Reasoning Architecture

La segunda encarnación es el motor de reflexión cognitiva para el módulo TEC Bii de Tecnológico de Monterrey. En este contexto, N.O.R.A. analiza el estado operacional de proyectos, empleados y briefs, generando reflexiones narrativas con patrones detectados y recomendaciones estratégicas.

Archivo	src/lib/tec-bii/nora-reflection.ts
Sprint	T2-6 — N.O.R.A. TEC Bii Reflection Engine
Tipo	Server-side Engine (Groq LLM)
Almacenamiento	users/{uid}/tec_bii_reflexiones

3. Pipeline Observer — Sprints F-3 y H-3

El Pipeline Observer es la capa de sensores más granular de N.O.R.A. Implementado en `src/core/pipeline-observer.ts`, registra cada request procesado por el pipeline de SOFIAA con métricas de performance y clasificación semántica.

3.1 Estructura de Evento

id	string — ID único generado (timestamp + random suffix)
timestamp	number — Unix ms del evento
messageSnippet	string — primeros 60 chars del mensaje del usuario
taskType	TaskType 'unknown' — clasificación semántica
provider	string — 'groq' 'gemini' 'none'
confidence	number 0–1 — confianza del clasificador de tareas
cacheLayer	'exact' 'semantic' 'miss'
latencyMs	number — tiempo total desde envío hasta último chunk

3.2 Persistencia Dual-Write (Sprint H-3)

En la Sprint H-3, el Pipeline Observer fue extendido para persistir eventos en Firestore además de localStorage. Esto permite a N.O.R.A. (NoraPanel) leer datos de todos los usuarios, no solo del dispositivo activo.

- ▶ Almacenamiento local: localStorage con clave 'sofiaa_pipeline_log', versión 'f3.1', máximo 50 eventos
- ▶ Almacenamiento remoto: `users/{uid}/pipeline_events/{eventId}` en Firestore
- ▶ Campo adicional 'date': YYYY-MM-DD derivado del timestamp, para queries de N.O.R.A.
- ▶ `setPipelineUserId(uid)`: función de inicialización llamada tras login

```
// Ruta Firestore de cada evento
doc(db, 'users', userId, 'pipeline_events', event.id)

// Campos persistidos
{
  id, timestamp, messageSnippet,
  taskType, provider, confidence,
  cacheLayer, latencyMs,
  date: '2026-07-24' // para filtros temporales
}
```

3.3 Estadísticas Agregadas

La función `getPipelineStats()` computa métricas agregadas sobre el log local para el panel de telemetría:

total	número total de eventos en log
cacheHitRate	$(\text{exactHits} + \text{semanticHits}) / \text{total}$ — tasa de caché global
exactHits	eventos con <code>cacheLayer === 'exact'</code>
semanticHits	eventos con <code>cacheLayer === 'semantic'</code>
llmCalls	eventos con <code>cacheLayer === 'miss'</code> (requirieron LLM)
avgLatencyMs	latencia promedio redondeada en ms
taskDistribution	<code>Record<TaskType, count></code> — histograma de tipos de tarea
providerDistribution	<code>Record<string, count></code> — histograma de proveedores

3.4 CacheLayer: Las Tres Capas de Eficiencia

- ▶ exact: hit exacto en caché — respuesta idéntica reutilizada sin LLM
- ▶ semantic: hit semántico — respuesta similar recuperada por embeddings
- ▶ miss: sin caché — llamada LLM completa (Groq o Gemini)

4. Long-term Memory — Sprint H-2

La memoria a largo plazo de SOFIAA OS es una abstracción de persistencia en texto plano que implementa un patrón dual-write: localStorage para acceso inmediato sin latencia y Firestore para sincronización multi-dispositivo.

4.1 Arquitectura Dual-Write

Capa primaria	localStorage — clave 'sofiaa_long_memory'
Capa secundaria	Firestore — users/{uid}/memory/long_term
Campo Firestore	'content' (string) + 'updatedAt' (serverTimestamp)
Fuente de verdad	Firestore — gana en conflicto al sincronizar
Modo offline	localStorage disponible sin conexión

4.2 API del Memory Store

```
// 1. Al login — sincronizar desde Firestore
await syncMemoryFromFirestore(userId);

// 2. Escribir memoria completa
writeLongMemory(userId, content);

// 3. Leer desde localStorage (sin latencia)
const memory = readLongMemory();

// 4. Añadir al final
appendLongMemory(userId, addition);

// 5. Borrar completamente
await clearLongMemory(userId);
```

4.3 Flujo de Sincronización

El mecanismo de sincronización prioriza Firestore como fuente de verdad multi-dispositivo:

- ▶ Si Firestore tiene contenido: descarga a localStorage (Firestore gana)
- ▶ Si Firestore está vacío pero localStorage tiene contenido: sube localStorage a Firestore
- ▶ Si ambos vacíos: inicia sesión con memoria en blanco
- ▶ Errores de sync: silenciados — fallback transparente a localStorage

4.4 Memory Timeline (Sprint 2.6)

Complementando el Memory Store, el módulo src/core/memory.summary.ts analiza las conversaciones para generar metadata estructurada para el timeline de sesiones:

generateSessionTitle()	primer mensaje del usuario, truncado a 60 chars
extractTags()	8 categorías temáticas detectadas por keywords (Producción, SOFIAA, PASCALL, Abraham, Servicios, Contacto, Aprendizaje, Decisión)
detectTopGoal()	6 tipos de objetivo detectados (Contratar, Contactar, Decidir, Comparar, Aprender, Informarse)

5. Perfil Cognitivo del Usuario — Sprint M-3

El CognitiveProfile es el modelo mental persistente que N.O.R.A. construye sobre cada usuario a lo largo del tiempo. Almacenado en Firestore bajo `users/{uid}/cognitive_profile/v1`, este documento evoluciona sesión a sesión mediante señales detectadas determinísticamente en los mensajes del usuario.

5.1 Esquema del Perfil

<code>uid</code>	string — UID del usuario propietario del perfil
<code>preferredDepth</code>	'concise' 'balanced' 'deep' — profundidad de respuesta preferida
<code>depthConfidence</code>	number 0–1 — confianza acumulada en la preferencia de profundidad
<code>formalityScore</code>	number 0–1 — 0.0=casual, 1.0=formal. Default: 0.5
<code>topicAffinity</code>	Record<string, number> — mapa tema→score, ej: {trabajo: 0.8}
<code>sessionCount</code>	number — sesiones de chat registradas
<code>lastActiveAt</code>	number — timestamp ms de última sesión activa
<code>createdAt</code>	number — timestamp ms de creación
<code>updatedAt</code>	number — timestamp ms de última actualización

Valores por defecto: preferredDepth='balanced', depthConfidence=0, formalityScore=0.5, topicAffinity={}, sessionCount=0

5.2 Señales Cognitivas (signals.ts)

El extractor de señales opera sobre los mensajes del usuario usando exclusivamente expresiones regulares — sin LLM, sin latencia, sin posibilidad de fallo. Detecta 5 tipos de señales:

<code>depth_increase</code>	17 patrones: 'más detalle', 'explica más', 'profundiza', 'amplía', 'a fondo', 'con detalle'. Confianza: 0.85
<code>depth_decrease</code>	15 patrones: 'más corto', 'en resumen', 'al grano', 'sin rollo', 'brevemente', 'sintetiza'. Confianza: 0.85
<code>formality_up</code>	11 patrones: 'por favor', 'usted', 'estimado/a', 'cordialmente', 'le solicito'. Confianza: 0.70
<code>formality_down</code>	16 patrones: 'wey', 'jaja', 'lol', 'xd', 'bro', 'chido', 'neta', 'chamba'. Confianza: 0.70
<code>topic_mention</code>	8 temas × múltiples keywords. Confianza: $\min(1, \text{matches} \times 0.30)$

Temas rastreados: trabajo, tecnología, comida, viaje, compras, salud, finanzas, creatividad.

5.3 Actualización del Perfil (profile.ts)

La función `updateCognitiveProfile()` aplica cada señal al perfil existente con deltas conservadores para asegurar convergencia gradual:

<code>depth_increase</code>	<code>depthConfidence += c×0.25</code> ; profundidad: concise→balanced→deep
<code>depth_decrease</code>	<code>depthConfidence += c×0.25</code> ; profundidad: deep→balanced→concise
<code>formality_up</code>	<code>formalityScore = min(1, score + c×0.08)</code>
<code>formality_down</code>	<code>formalityScore = max(0, score - c×0.08)</code>
<code>topic_mention</code>	<code>topicAffinity[topic] = min(1, prev + c×0.12)</code>

Decay de afinidad temática: 0.5% por sesión ($\times 0.995$). Temas con score < 0.01 son eliminados para prevenir fosilización.

5.4 Inyección en el System Prompt (profile.ts: buildCognitiveBlock)

El bloque cognitivo se inyecta en el system prompt de cada conversación si el perfil existe. El diseño es compacto para minimizar consumo de tokens:

```
[PERFIL COGNITIVO DEL USUARIO]
• Respuestas: detalladas y profundas — el usuario aprecia explicaciones completas
• Tono: natural y conversacional
• Temas de interés: tecnología, trabajo, creatividad
• Han pasado 5 días desde la última sesión — retoma con calidez.
Adapta tu estilo a este perfil de forma natural. No lo menciones explícitamente.
```

Lógica de tonalidad: formalityScore >0.65 → 'formal y profesional'; <0.35 → 'casual y cercano, como con un amigo'; else → 'natural y conversacional'.

La profundidad solo se menciona si depthConfidence >0.15, evitando assert prematuro sobre preferencias no establecidas.

6. CognitivePolicy Engine — Sprint D-D

El CognitivePolicy Engine es la capa declarativa de gobernanza del comportamiento de SOFIAA. Implementado en `src/core/cognitive.policy.ts`, define contratos semánticos que el LLM recibe como instrucciones y el evaluador verifica post-stream.

6.1 Flujo del Policy Engine

```
route.ts
→ getPoliciesForContext(ctx)    // filtrar políticas aplicables
→ buildPolicyBlock(policies)    // generar bloque para system prompt
→ (inyectar en system prompt)
→ LLM genera respuesta
→ evaluateResponse()           // detectar violaciones post-stream
→ violations al EventBus        // alertas asíncronas
```

6.2 Tipos Fundamentales

PolicyScope	'global' 'extension' 'goal_active' 'unauthenticated'
PolicyConstraint	id, instruction (para LLM), description (para evaluador), severity ('warn' 'error'), detect(response, context)
CognitivePolicy	id, name, scope, appliesTo[], priority (higher=first), constraints[]
PolicyContext	activePath?, extensionId?, userRole?, isGoalActive?, userMessage?

6.3 Registro de 8 Políticas

POLICY_TONE_SOFIAA (global, priority=100)

- ▶ no_robotic: detecta respuestas <3 palabras sin tokens de acción
- ▶ no_apologies: detecta 'lo siento mucho' / 'disculpe muchísimo'
- ▶ spanish_first (severity=error): detecta inglés cuando el usuario escribió en español

POLICY_PRIVACY (global, priority=95)

- ▶ no_expose_keys (severity=error): detecta sk-*, Bearer tokens, Alza* en respuestas
- ▶ no_expose_internal_paths: detecta /etc/, /var/, node_modules/, .env, process.env.

POLICY_NAVIGATION (global, priority=90)

- ▶ exact_routes (severity=error): detecta si hay más de un token [NAVIGATE:...] en la respuesta

POLICY_JP_MEMORIAL (extension /jp-memorial, priority=85)

- ▶ empathy_first (severity=error): detecta respuestas abruptas (<8 palabras sin léxico de empatía)
- ▶ no_price_pressure: detecta 'oferta especial', 'por tiempo limitado'

POLICY_GOAL_ACTIVE (goal_active, priority=92)

- ▶ one_question_per_step: detecta más de 2 signos de interrogación en modo goal activo
- ▶ confirm_step_data: instrucción inyectada (detección semántica diferida)

POLICY_UNAUTHENTICATED (unauthenticated, priority=88)

- ▶ gentle_auth_prompt: detecta 'debes iniciar sesión' o 'no puedes acceder'

POLICY_TEC_BI (extension /tec-bi, priority=80)

- ▶ formal_language: detecta 'wey', 'bro', 'chido', 'jaja' en contexto institucional

- ▶ data_grounded: detecta números grandes sin contexto de extensión activa

POLICY_MARKETING_SOFIA (extension /marketing-sofia, priority=80)

- ▶ actionable_ideas: detecta ideas vagas + respuesta corta (<3 líneas)

6.4 Resolución Contextual

```
getPoliciesForContext(ctx)
→ scope 'global'           → siempre incluida
→ scope 'extension'        → si ctx.activePath.startsWith(policy.appliesTo)
→ scope 'goal_active'      → si ctx.isGoalActive === true
→ scope 'unauthenticated' → si !ctx.userRole
→ ordenar por priority DESC
```

7. NoraPanel — Panel de Observabilidad (Sprint I-0)

NoraPanel.tsx es el componente React que materializa la observabilidad de N.O.R.A. para el administrador del sistema. Montado en page.tsx tras la activación por frase secreta, permite visualizar en tiempo real el estado de todos los usuarios y su actividad de pipeline.

7.1 Interfaces de Datos

```
interface UsuarioDoc {
  uid: string;
  email: string;
  nombre?: string;
  rol: 'admin' | 'director' | 'vp' | 'user';
  createdAt: Timestamp;
}

interface PipelineEvent {
  id: string;
  timestamp: number;
  messageSnippet: string;
  taskType: string; //
  conversational|data_query|creative|analytical|navigation|unknown
  provider: string;
  confidence: number;
  cacheLayer: string; // exact|semantic|miss
  latencyMs: number;
  date: string; // YYYY-MM-DD
}
```

7.2 Fuentes de Datos Firestore

Usuarios	collection(db, 'usuarios') — todos los documentos de usuario
Pipeline Events	collection(db, 'users', uid, 'pipeline_events') — por usuario
Long-term Memory	doc(db, 'users', uid, 'memory', 'long_term') — por usuario

7.3 Tarjetas de Estadísticas Globales

Usuarios Registrados	count(usuariosSnap.docs)
Total Interacciones	suma de todos los pipeline_events de todos los usuarios
Latencia Promedio	avgLatencyMs calculado sobre todos los eventos
Con Memoria Activa	usuarios que tienen el documento long_term en Firestore
Cache Hit Rate	(exact + semantic) / total × 100%

7.4 Las Tres Pestañas del Panel

Pestaña 1 — Usuarios

Lista todos los usuarios registrados con: avatar inicial, nombre/email, rol con color semántico (admin=ROSA, director=LILA, vp=AZUL, user=GRIS), fecha de registro.

Pestaña 2 — Últimos Eventos

Agrega y ordena todos los pipeline_events de todos los usuarios por timestamp descendente. Muestra para cada evento: mensaje snippet, tipo de tarea con color, proveedor LLM, capa de caché, latencia en ms, fecha.

conversational	color: AZUL
data_query	color: LILA
creative	color: ROSA
analytical	color: WARN (amarillo)
navigation	color: SUCCESS (verde)

Pestaña 3 — Distribución

Histogramas visuales de dos dimensiones del sistema:

- Distribución de TaskType: barras proporcionales con porcentaje y conteo por tipo de tarea
- Distribución de CacheLayer: barras de exact/semantic/miss con porcentajes para evaluar eficiencia del caché

7.5 Paleta de Colores N.O.R.A.

DARK	#09090F — fondo del panel, headers oscuros
LILA	#A855F7 — identidad primaria N.O.R.A.
AZUL	#60A5FA — tareas conversacionales, VP
ROSA	#F472B6 — tareas creativas, admin
SUCCESS	#34D399 — caché exacto, éxito
WARN	#FBBF24 — tareas analíticas, alertas

8. Mecanismo de Activación 'Fiat Lux'

El panel N.O.R.A. no es accesible directamente por URL. Se activa mediante un mecanismo de frase secreta integrado en page.tsx, la interfaz principal de SOFIAA. Este diseño implementa seguridad por oscuridad: el panel de observabilidad solo es visible para quien conoce la activación.

8.1 Flujo de Activación

```
// En page.tsx:  
// 1. Usuario escribe la frase secreta en el chat  
// 2. Sistema detecta la frase  
// 3. Estado noraActive se establece en true  
// 4. NoraPanel se monta como overlay sobre la interfaz principal  
// 5. Panel carga datos de Firestore en tiempo real
```

8.2 Filosofía de Diseño

La decisión de no exponer NoraPanel como ruta pública (/admin/nora o similar) responde a principios de seguridad y diseño operacional:

- ▶ Zero exposure: sin ruta, sin menú, sin link — invisible para usuarios no autorizados
- ▶ Context-sensitive: el panel emerge dentro del contexto del chat, no como página separada
- ▶ Revocable: la frase puede ser cambiada sin modificar rutas ni desplegar

Referencia histórica: El nombre 'Fiat lux' (latín: 'hágase la luz') evoca el momento en que el observador tiene visibilidad total del sistema — el primer acto de creación.

9. N.O.R.A. en TEC Bii — Motor de Reflexión (Sprint T2-6)

En el contexto del módulo TEC Bii (Tecnológico de Monterrey), N.O.R.A. actúa como motor de reflexión institucional. Implementado en `src/lib/tec-bii/nora-reflection.ts`, este componente sintetiza el estado operacional de proyectos, empleados y briefs en una narrativa estructurada con patrones detectados.

9.1 Estructura NoraReflection

<code>id</code>	string — identificador único de la reflexión
<code>narrativa</code>	string — texto generativo del estado operacional
<code>patrones[]</code>	NoraPattern[] — lista de patrones detectados
<code>recomendacion</code>	string — acción estratégica sugerida
<code>estadoGeneral</code>	'crítico' 'alerta' 'estable' 'óptimo'
<code>generadoEn</code>	Timestamp — fecha de generación
<code>contexto</code>	objeto con <code>proyectosAnalizados</code> , <code>empleadosAnalizados</code> , <code>briefsAnalizados</code> , <code>hipotesisActivas</code>

9.2 Los 6 Patrones de Detección

<code>carga</code>	nivel de ocupación del equipo detectado en datos de empleados
<code>urgencia</code>	distribución de urgencia en proyectos activos
<code>talento</code>	análisis de disponibilidad y capacidad del equipo
<code>oportunidad</code>	briefs pendientes con alto <code>briefScore</code> sin asignación
<code>riesgo</code>	proyectos o empleados con <code>alertaRiesgo=true</code>
<code>tendencia</code>	dirección de calidad y cumplimiento en evaluaciones recientes

9.3 Parámetros del Motor

<code>COOLDOWN_HOURS</code>	6h — tiempo mínimo entre reflexiones por usuario
<code>Groq params</code>	600 tokens, temperatura 0.65, json:true
<code>Datos leídos</code>	20 proyectos + 15 empleados + 15 briefs en paralelo
<code>Almacenamiento</code>	<code>users/{uid}/tec_bii_reflexiones/{reflexionId}</code>
<code>getNoraReflections</code>	limit=5 reflexiones más recientes

9.4 buildContextString — Datos para el LLM

La función `buildContextString` construye una cadena estructurada con el contexto operacional para el modelo LLM:

```
CONTEXTO TEC BII:
PROYECTOS EN PRODUCCIÓN: proyecto A (urgencia: 0.92), proyecto B (urgencia: 0.71)
PROYECTOS EN REVISIÓN: proyecto C
PROYECTOS URGENTES: proyecto D (deadline en 2 días)

EQUIPO:
  Saturados (carga ≥0.85): Juan Pérez (0.93), Ana Torres (0.87)
  Disponibles (carga <0.35): Carlos López (0.22)
  Promedio de carga: 0.64

BRIEFS PENDIENTES: 3 (briefScore promedio: 78)
HIPÓTESIS NEXO ACTIVAS: 7
```

9.5 Integración con el Refine CRON (Sprint M-7)

La reflexión N.O.R.A. de TEC Bii es el segundo paso del batch de refinamiento cognitivo ejecutado por `refine.ts`. El flujo completo por usuario es:

- ▶ Fase 1: republishStaleEntities — republicar entidades con nexoNodeIid vacío o lastCognitiveSync >24h
- ▶ Fase 2: runNoraReflection(uid, force=false) — reflexión N.O.R.A. con cooldown de 6h
- ▶ Fase 3: runCrossDomainAnalysis — análisis de hipótesis cruzadas NEXO ↔ TEC Bii

10. Agent Runtime y el Analyze Endpoint — Sprints G-1/G-2/G-3/H-1

N.O.R.A. opera también como orquestador del razonamiento multi-paso de SOFIAA a través del Agent Runtime. Implementado bajo el paradigma ReAct (Reasoning + Acting), este motor permite a SOFIAA planificar y ejecutar tareas complejas usando herramientas especializadas.

10.1 Agent Tools (Sprint G-2)

query_data	Consulta datos estructurados de Firestore para el contexto del usuario
search_memory	Busca en la memoria larga del usuario y en los nodos NEXO
analyze	Llama a /api/analyze con datos + instrucción para análisis especializado

10.2 Endpoint /api/analyze (Sprint H-1)

El endpoint /api/analyze fue creado para poder llamarse dentro del loop ReAct sin recursión con /api/chat. Acepta datos crudos e instrucción de análisis, devuelve análisis conciso:

Método	POST
Body	{ data: string, instruction: string }
LLM	Groq — maxTokens=512, temperatura=0.3
Límite de datos	3000 chars (truncado en el prompt)
Límite respuesta	300 palabras, sin preámbulos
Runtime	Node.js — separado del stream principal

```
// Prompt template de /api/analyze
Eres un analista de datos experto. Analiza el siguiente contenido
y responde la instrucción de forma concisa y directa.

DATOS: ${data.slice(0, 3000)}
INSTRUCCIÓN: ${instruction}

Responde en español, máximo 300 palabras, sin preámbulos.
```

10.3 Separación de Responsabilidades

La existencia de /api/analyze como endpoint independiente de /api/chat es una decisión arquitectónica fundamental que refleja la filosofía de N.O.R.A.:

- ▶ Conteo de tokens separado: el análisis interno no contamina el contexto de la conversación principal
- ▶ Sin recursión: el loop ReAct puede llamar /api/analyze sin riesgo de llamadas circulares
- ▶ Especialización: parámetros optimizados para análisis (temperatura baja, prompt estructurado)

11. Flujo de Datos Completo de N.O.R.A.

El siguiente flujo muestra cómo N.O.R.A. observa, procesa y retroalimenta cada interacción del usuario con SOFIAA:

11.1 Flujo por Request (Runtime)

```
Usuario envía mensaje
├─ 1. getCognitiveProfile(uid)           // leer perfil Firestore
├─ 2. getPoliciesForContext(ctx)         // resolver políticas activas
├─ 3. buildCognitiveBlock(profile)       // generar bloque cognitivo
├─ 4. buildPolicyBlock(policies)         // generar bloque de políticas
├─ 5. getSemanticNexoContext(uid, query) // contexto NEXO (semántico)
├─ 6. [inyectar todo en system prompt]
├─ 7. LLM genera respuesta (stream)
└─ POST-STREAM (async, sin bloquear):
    ├─ extractSignals(userMessages)     // detectar señales cognitivas
    ├─ updateCognitiveProfile(uid, sigs) // actualizar perfil
    ├─ attendNexoNodes(uid, nodeIds)     // reforzar nodos usados
    └─ recordPipelineEvent(...)           // log telemetría
```

11.2 Flujo de Telemetría (Client-side)

```
recordPipelineEvent(event)
├─ writeStore(localStorage)             // inmediato, sin latencia
└─ pushEventToFirestore(userId, event)  // async, best-effort
    └─ doc: users/{uid}/pipeline_events/{event.id}
```

11.3 Flujo de Sincronización al Login

```
Usuario hace login
├─ setPipelineUserId(uid)               // activar escritura a Firestore
└─ syncMemoryFromFirestore(userId)
    ├─ Firestore tiene contenido → localStorage = Firestore content
    └─ Firestore vacío + localStorage tiene contenido → push a Firestore
```

11.4 Flujo CRON Diario (TEC Bii)

```
CRON /api/tec-bii/refine (daily)
├─ discoverTecBiiUsers()                // collectionGroup discovery
└─ por cada usuario:
    ├─ republishStaleEntities()          // Fase 1
    ├─ runNoraReflection(uid)           // Fase 2 (N.O.R.A.)
    └─ runCrossDomainAnalysis()          // Fase 3
```


12. Observabilidad y Métricas del Sistema

N.O.R.A. como sistema de observabilidad define un conjunto de métricas que permiten al administrador entender el estado de salud del pipeline de SOFIAA y la evolución cognitiva de los usuarios.

12.1 Métricas de Pipeline

Cache Hit Rate	KPI primario de eficiencia. Objetivo: >60% (evitar LLM innecesarios)
Latencia Promedio	Salud del pipeline. Groq típicamente <800ms; Gemini 1–3s
LLM Calls / Total	Proporción de misses. Alta = falta cobertura semántica en caché
TaskType Distribution	Perfil de uso: mayoría conversacional indica uso general; data_query indica uso avanzado
Provider Distribution	Balance entre Groq y Gemini; detecta fallos de fallback

12.2 Métricas Cognitivas

depthConfidence	Velocidad de aprendizaje de preferencias. >0.5 = perfil maduro
formalityScore	Indica cultura de comunicación del usuario (0=muy casual, 1=muy formal)
topicAffinity	Mapa de intereses acumulados — útil para personalización proactiva
sessionCount	Antigüedad del usuario en el sistema
COGNITIVE_RETURN_THRESHOLD_DAYS=3	Días sin actividad antes de saludo de retorno

12.3 Métricas de Reflexión TEC Bii

estadoGeneral	Estado consolidado: crítico/alerta/estable/óptimo
patrones detectados	Cuenta y tipo de patrones en cada reflexión
cooldown 6h	Garantiza al menos 4 reflexiones/día en operación continua
hipótesisActivas	Nodos de razonamiento cruzado NEXO ↔ TEC Bii activos

13. Análisis Arquitectónico y Decisiones de Diseño

13.1 Zero-Failure Design en el Cognitive Engine

Una decisión arquitectónica crítica en N.O.R.A. es que el Cognitive Engine nunca debe romper el flujo de chat. Esto se implementa en tres niveles:

- ▶ `getCognitiveProfile()`: retorna null en caso de error — el chat continúa sin perfil
- ▶ `updateCognitiveProfile()`: try/catch total — silencia cualquier error de escritura
- ▶ `extractSignals()`: regex determinístico — sin LLM, sin posibilidad de timeout

13.2 Asincronía Post-Stream

Todas las operaciones de actualización de N.O.R.A. ocurren después de que el stream de respuesta se ha completado, implementando el patrón 'fire-and-forget':

```
// En route.ts (post-stream)
waitUntil(Promise.all([
  updateCognitiveProfile(uid, signals),
  attendNexoNodes(uid, nodeIds),
  recordPipelineEvent(metrics),
]))
```

Esto garantiza que el usuario recibe la respuesta con latencia mínima mientras N.O.R.A. actualiza su modelo en paralelo.

13.3 Separación entre Observación y Actuación

N.O.R.A. mantiene una separación estricta entre sus funciones de observación y actuación:

Observación (N.O.R.A.)	Pipeline Observer, CognitiveProfile, NoraPanel — solo leen y registran
Actuación (SOFIAA)	System prompt injection, policy evaluation — actúan sobre la respuesta
Reflexión (N.O.R.A.)	nora-reflection.ts, runReflection() — sintetizan observaciones en insights

13.4 Evolución de Sprints

La arquitectura de N.O.R.A. siguió un patrón de estratificación progresiva:

Sprint F-3	Pipeline Observer en localStorage (telemetría local)
Sprint H-2	Long-term Memory Store con dual-write
Sprint H-3	Pipeline Observer extendido a Firestore (telemetría distribuida)
Sprint I-0	NoraPanel — primera visualización admin del sistema
Sprint M-3	CognitiveProfile — modelo mental del usuario
Sprint D-D	CognitivePolicy Engine — gobernanza declarativa
Sprint G-1/2/3	Agent Runtime — razonamiento multi-paso (ReAct)
Sprint H-1	Analyze Endpoint — herramienta especializada de análisis
Sprint T2-6	N.O.R.A. reflexión TEC Bii — observación institucional

14. Conclusión Doctoral

N.O.R.A. — Neural Observer & Reasoning Architecture — representa una de las contribuciones más sofisticadas del ecosistema SOFIAA OS. Su arquitectura distribuida, que abarca desde sensores de pipeline de microsegundos hasta reflexiones narrativas institucionales de largo plazo, implementa en la práctica el concepto de metacognición aplicada a sistemas de IA.

14.1 Contribuciones Técnicas

- ▶ CognitiveProfile: primer modelo de usuario persistente en SOFIAA OS, con señales determinísticas y evolución sesión a sesión
- ▶ CognitivePolicy Engine: sistema declarativo de gobernanza del comportamiento LLM con evaluación post-stream
- ▶ Pipeline Observer: telemetría de grano fino con dual-write localStorage + Firestore
- ▶ Long-term Memory: sincronización multi-dispositivo con Firestore como fuente de verdad
- ▶ NoraPanel: panel de observabilidad en tiempo real activado por frase secreta
- ▶ N.O.R.A. TEC Bii: reflexión institucional generativa con 6 patrones y 4 estados operacionales

14.2 Patrones de Diseño Evidenciados

Observer Pattern	Pipeline Observer registra eventos sin interferir en el flujo principal
Dual-Write Consistency	localStorage + Firestore para disponibilidad y consistencia
Zero-Failure Cognitive	try/catch total en el cognitive engine — chat nunca se rompe
Post-Stream Async	waitUntil() para operaciones no bloqueantes post-respuesta
Declarative Governance	CognitivePolicy separa instrucciones del código de detección
Progressive Enhancement	cada sprint añade una capa sin romper la anterior

14.3 Visión a Futuro

El diseño actual de N.O.R.A. anticipa varias extensiones naturales documentadas en los comentarios del código:

- ▶ Fase 2 del CognitiveProfile: refinamiento periódico con LLM para análisis semántico de señales (en lugar de solo regex)
- ▶ N.O.R.A. reflexión NEXO: extensión del motor de reflexión TEC Bii al grafo N.E.X.O. personal del usuario
- ▶ Dashboard de Salud Cognitiva: visualización longitudinal de la evolución del CognitiveProfile
- ▶ Policy Learning: políticas que evolucionan basadas en violaciones detectadas

Síntesis doctoral: N.O.R.A. no es simplemente un sistema de logging o monitoreo. Es la capa de autoconciencia que permite a SOFIAA OS aprender de sus propias interacciones, adaptarse a cada usuario y gobernar su comportamiento de forma declarativa. En términos cognitivos: es la metacognición del sistema.

N.O.R.A.

SOFIAA OS — Sistema Cognitivo Autónomo
2026 · Clasificación: Arquitectura Interna Confidencial